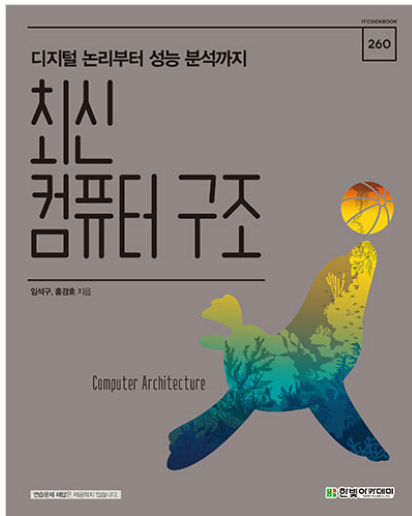


# 컴퓨터구조 Computer Architecture

1장 – 서론  
김현석

# 수업계획서 요약



- 교과목명: 컴퓨터구조 (Computer Architecture)
- 영역: 전공기초
- 학점/시간: 3학점/3시간: O2O 강의(오프라인 수업: 목요일)
- 담당교수: 김현석
- 주교재: 최신 컴퓨터 구조 (한빛아카데미 출판사)
- 학습평가방법
  - 출석(10%): 전체 출석일 수의 2/3 이상 출석  
(온/오프라인 각 1회 결석시 0.5점 감점)
  - 과제(10%): 설계 및 자료조사
  - 중간시험(40%): 지필고사
  - 기말시험(40%): 지필고사
- 보강안내
  - 6월6일(목) 현충일 → 6월14일(금)

# 컴퓨터구조를 왜? 배워야 하나요?

## 실력 있는 개발자가 되기 위해서



User

VS.



Developer

# Developer





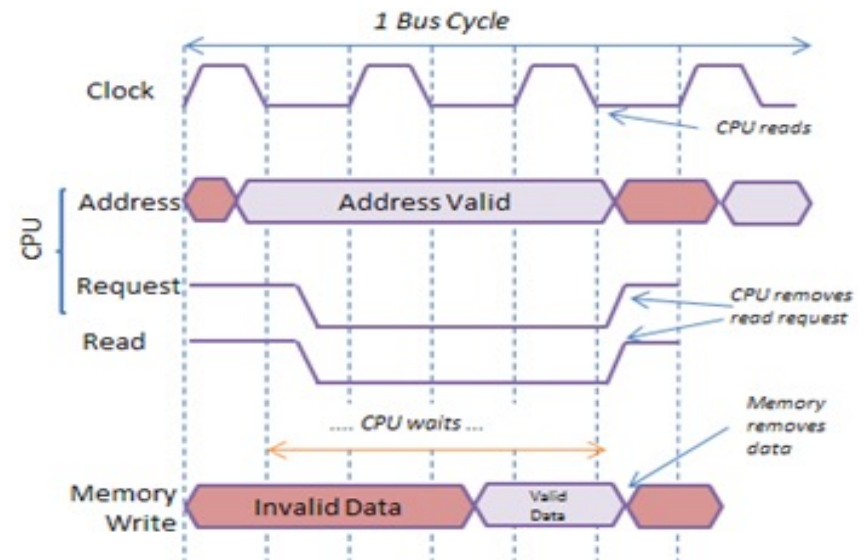
# 컴퓨터구조를 배울 때 제일 먼저 이해해야 하는 개념

## 디지털 컴퓨터는 “클럭(Clock)”에 동기 되어 움직인다.



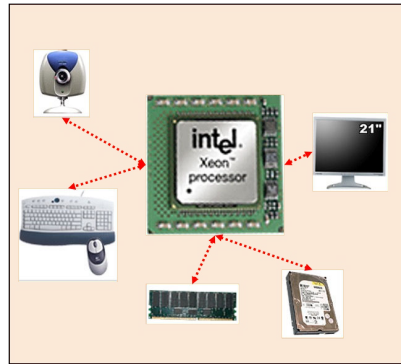
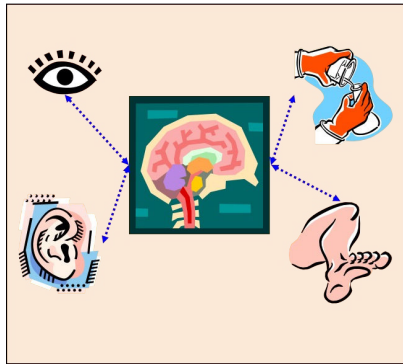
인체의 모든 기관은 심장박동에 따라 작동함

VS.

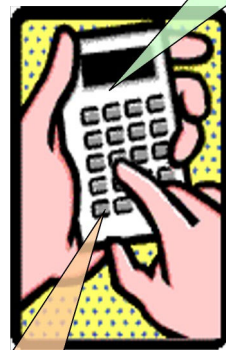


컴퓨터의 모든 장치도 클럭에 따라 작동함

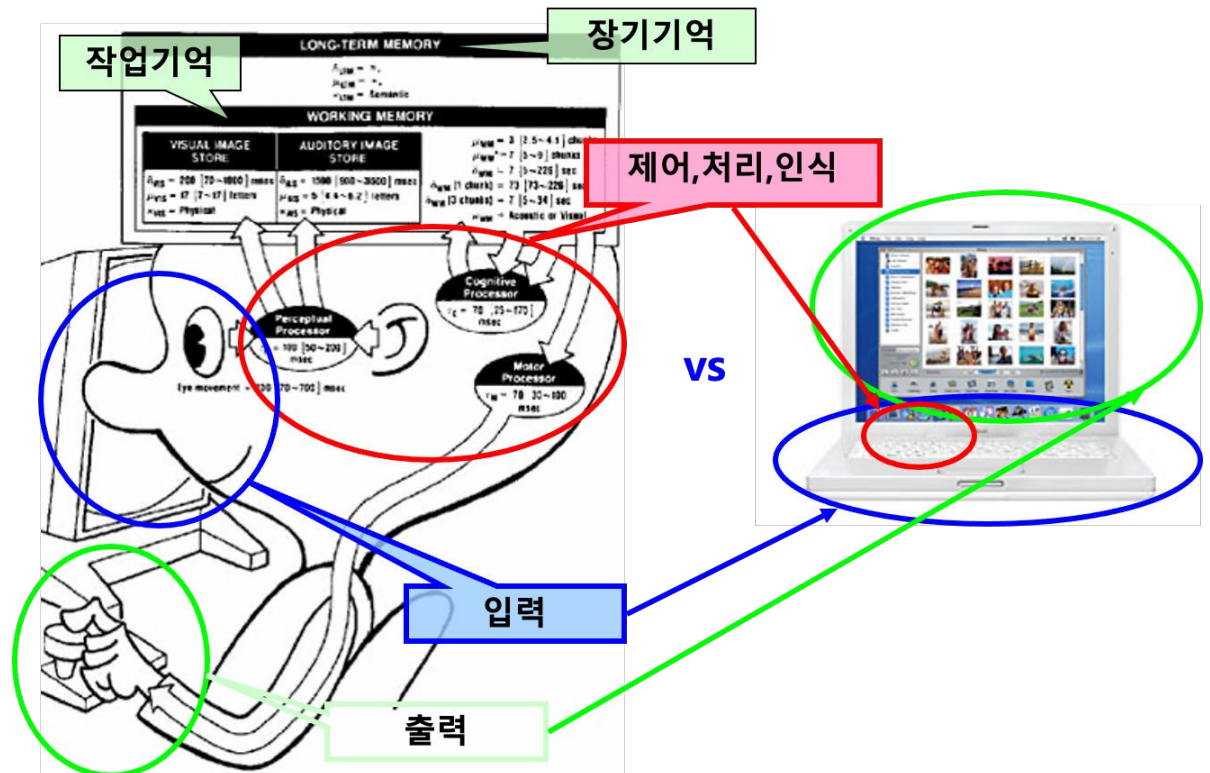
# 인간과 컴퓨터



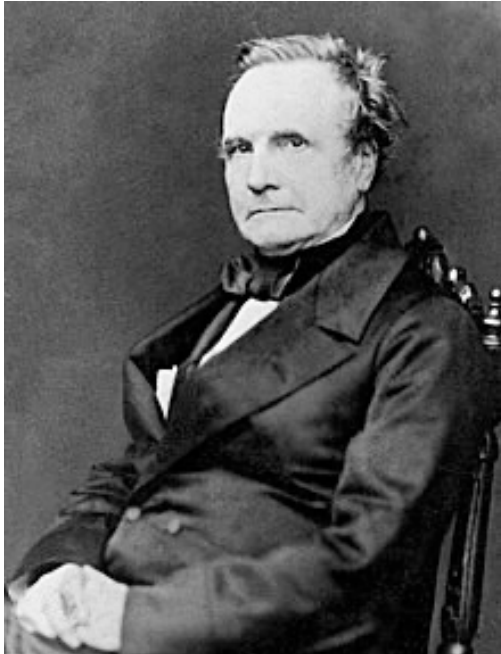
vs



컴퓨터 H/W  
키보드, 액정



# 1833년. 순차적으로 방정식을 풀 수 있는 (과거) 기계식 컴퓨터



Charles Babbage

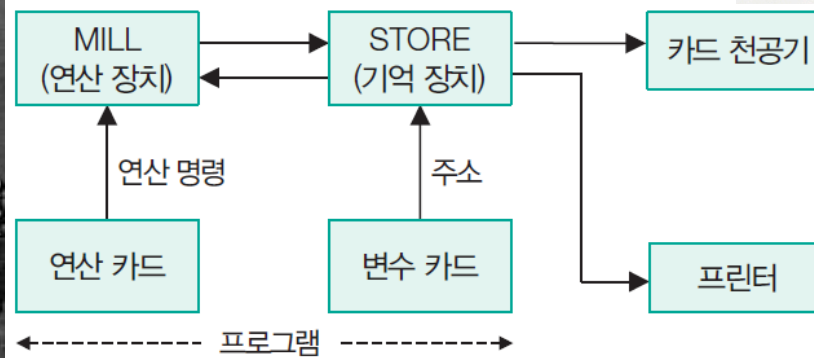
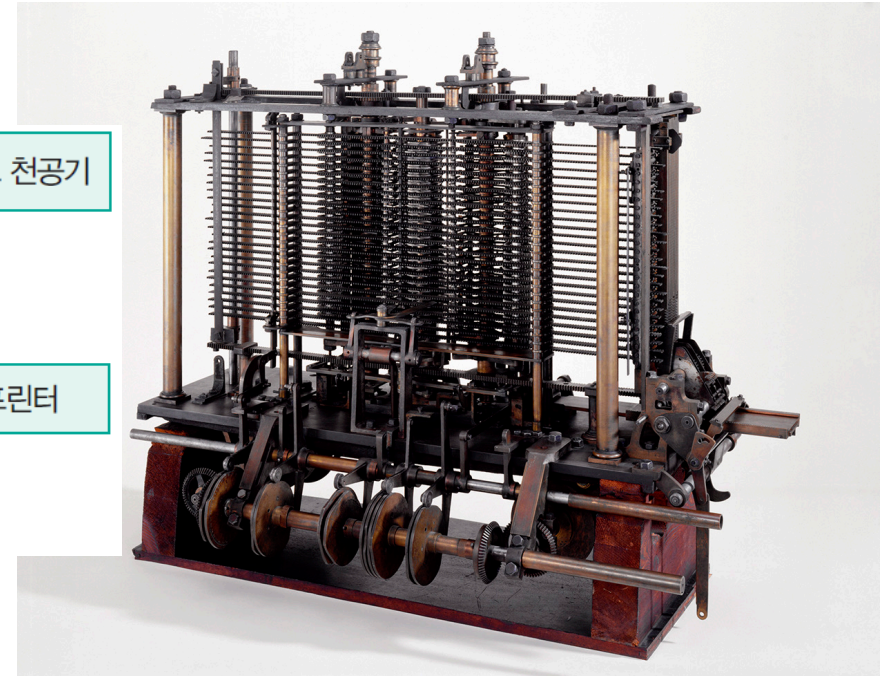


그림 1-9 해석 기관의 기본 구조

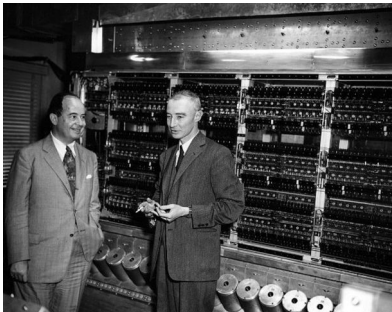


3종류의 천공카드로 입력:  
수학 연산 카드, 숫자 변수 카드, 저장용 카드

# 1945년 (현대) 컴퓨터 구조에 대한 생각



John von Neumann



맨해튼 프로젝트 멤버

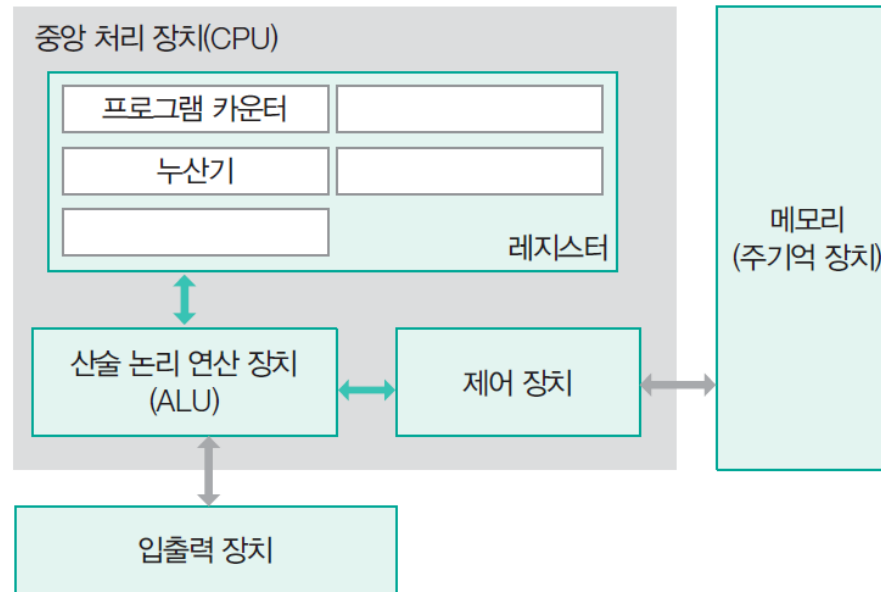


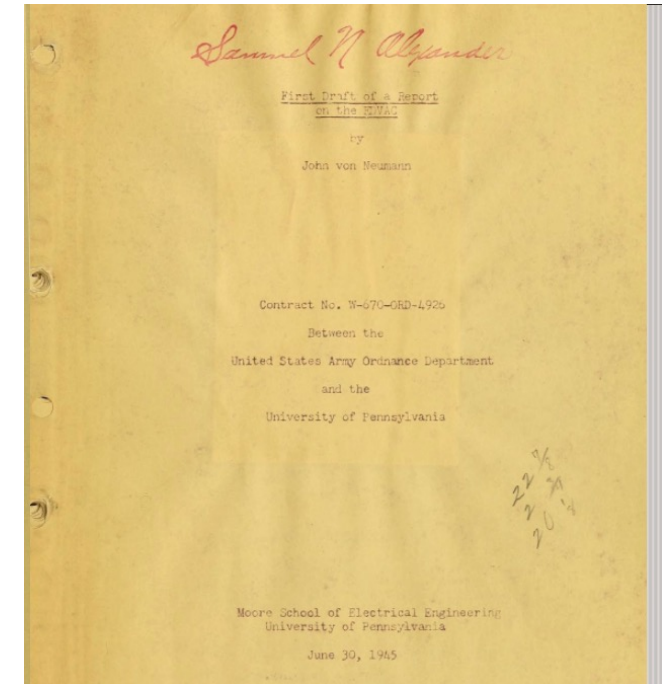
그림 1-12 폰 노이만 구조

프로그램을 저장해서  
실행하는 구조

**Von Neumann Privately Circulates the First Theoretical Description of a Stored-Program Computer: the von Neumann Architecture**

6/30/1945

[Permalink](#)



<https://historyofinformation.com/detail.php?id=644>



# 1833 vs. 1945 vs. 현재 → 인간이기에 생각한 구조가 같음

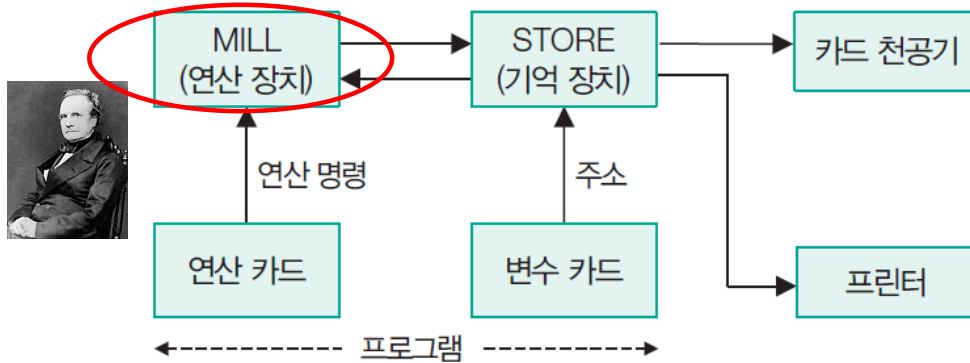


그림 1-9 해석 기관의 기본 구조

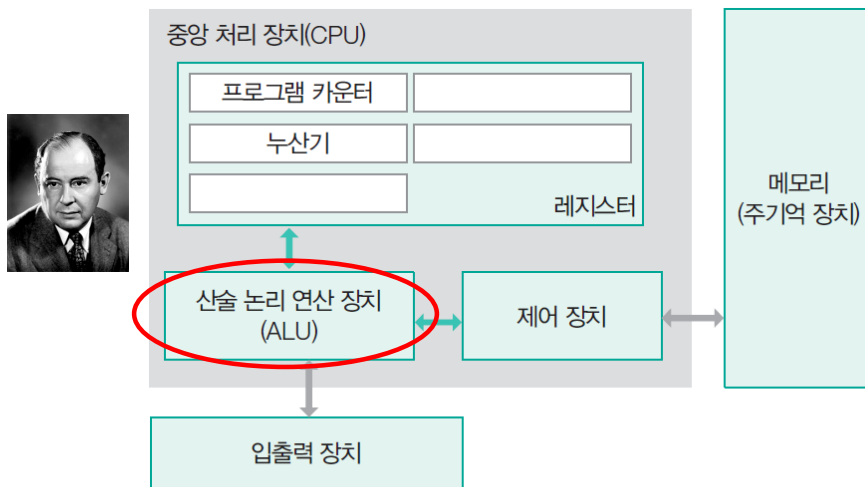
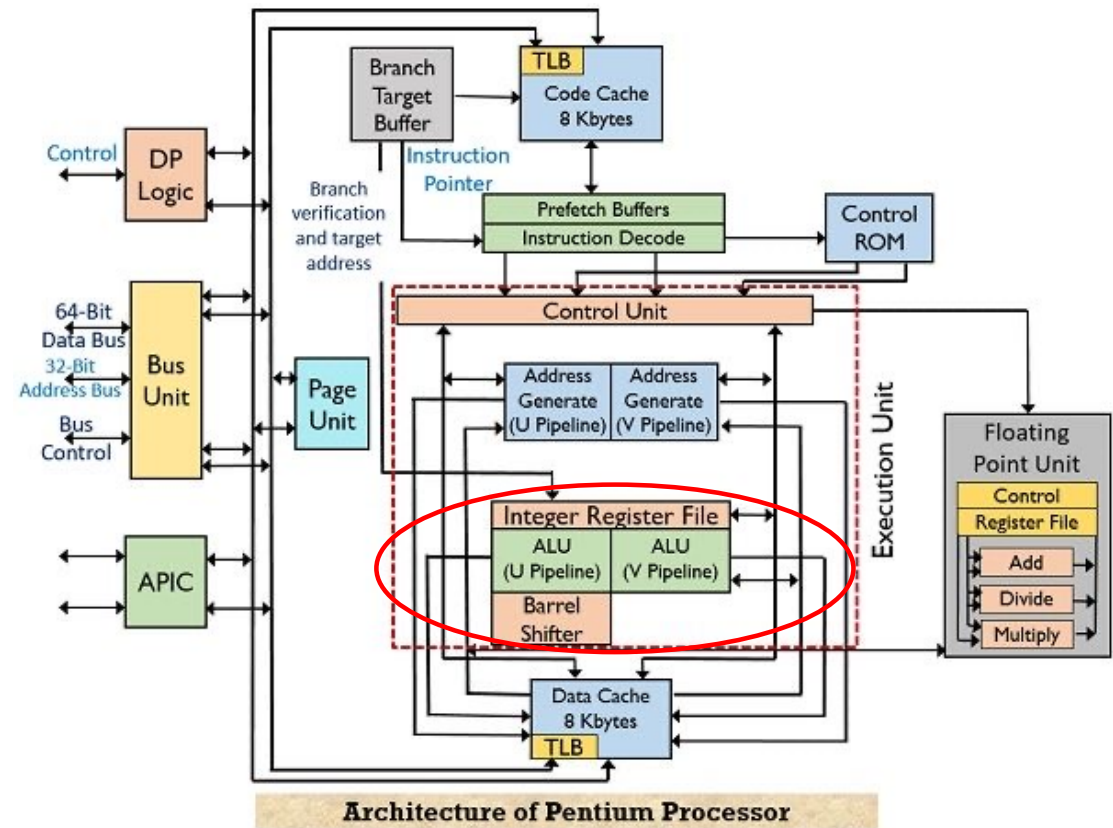
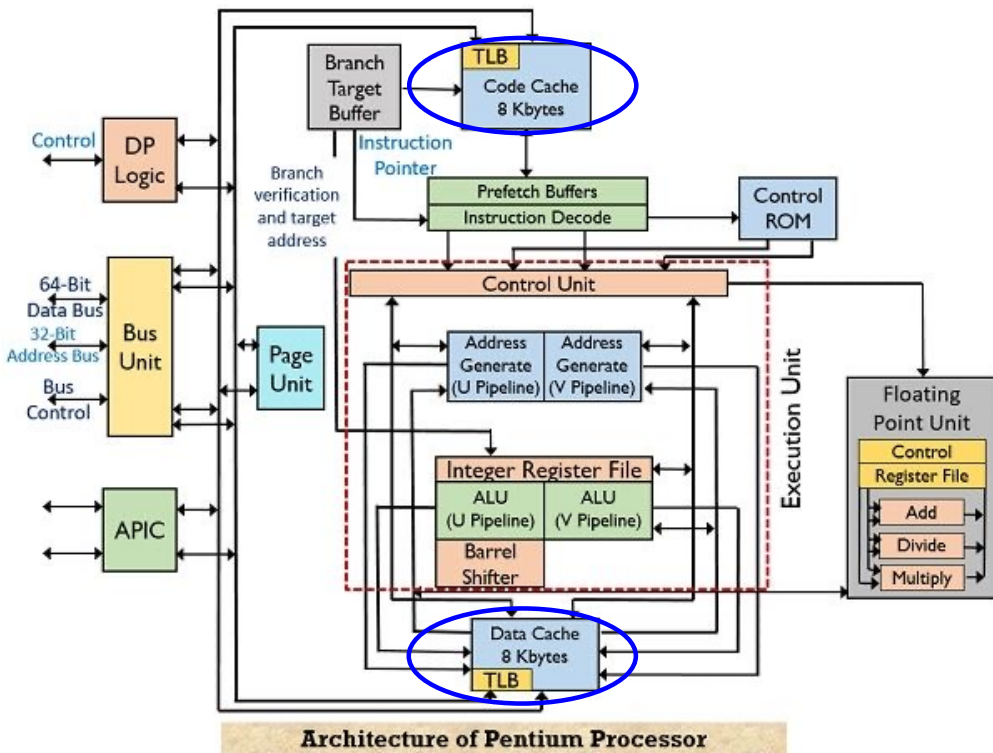


그림 1-12 폰 노이만 구조



Intel Pentium 구조

옛날 CPU → 최신 CPU  
 옛날 프로그래밍 = 최신 프로그래밍 ?  
 → 컴퓨터구조를 배워야 함!



Architecture of Pentium Processor

Cache 메모리가 있는 CPU  
 Cache의 Locality 특성을 이용

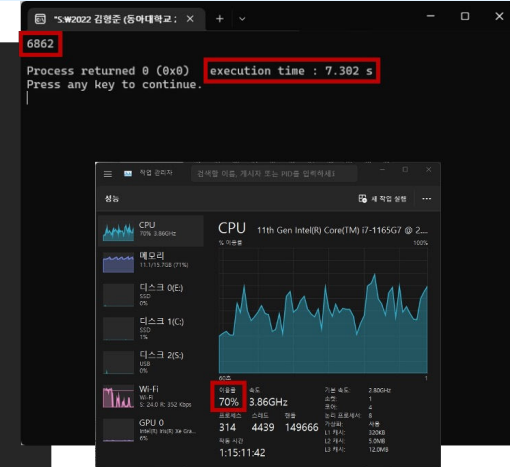
```
#include <chrono>
#include <iostream>

using namespace std;

#define SIZE 20240

int matrix[SIZE][SIZE] = {};

int main()
{
    auto start = chrono::high_resolution_clock::now();
    for(int i=0; i<SIZE; ++i) {
        for(int j=0; j<SIZE; ++j) {
            matrix[j][i] = 1;
        }
    }
    auto end = chrono::high_resolution_clock::now();
    auto elapsed = end - start;
    auto elapsedTime =
        chrono::duration_cast<chrono::milliseconds>(elapsed);
    cout << elapsedTime.count() << endl;
    return 0;
}
```



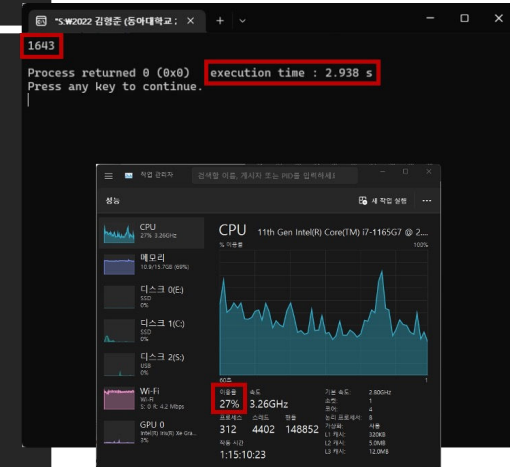
```
#include <chrono>
#include <iostream>

using namespace std;

#define SIZE 20240

int matrix[SIZE][SIZE] = {};

int main()
{
    auto start = chrono::high_resolution_clock::now();
    for(int i=0; i<SIZE; ++i) {
        for(int j=0; j<SIZE; ++j) {
            matrix[i][j] = 1;
        }
    }
    auto end = chrono::high_resolution_clock::now();
    auto elapsed = end - start;
    auto elapsedTime =
        chrono::duration_cast<chrono::milliseconds>(elapsed);
    cout << elapsedTime.count() << endl;
    return 0;
}
```



# 컴퓨터 시스템의 구성

## □ 컴퓨터의 기본 구성

- **하드웨어** : 컴퓨터에서 각종 정보를 입력하여 처리하고 저장하는, 동작이 실제 일어나게 하는 물리적인 실체
- **소프트웨어** : 정보 처리의 종류를 지정하고 정보의 이동 방향을 결정하는, 동작이 일어나는 시간을 지정해 주는 명령(command)들의 집합(프로그램)

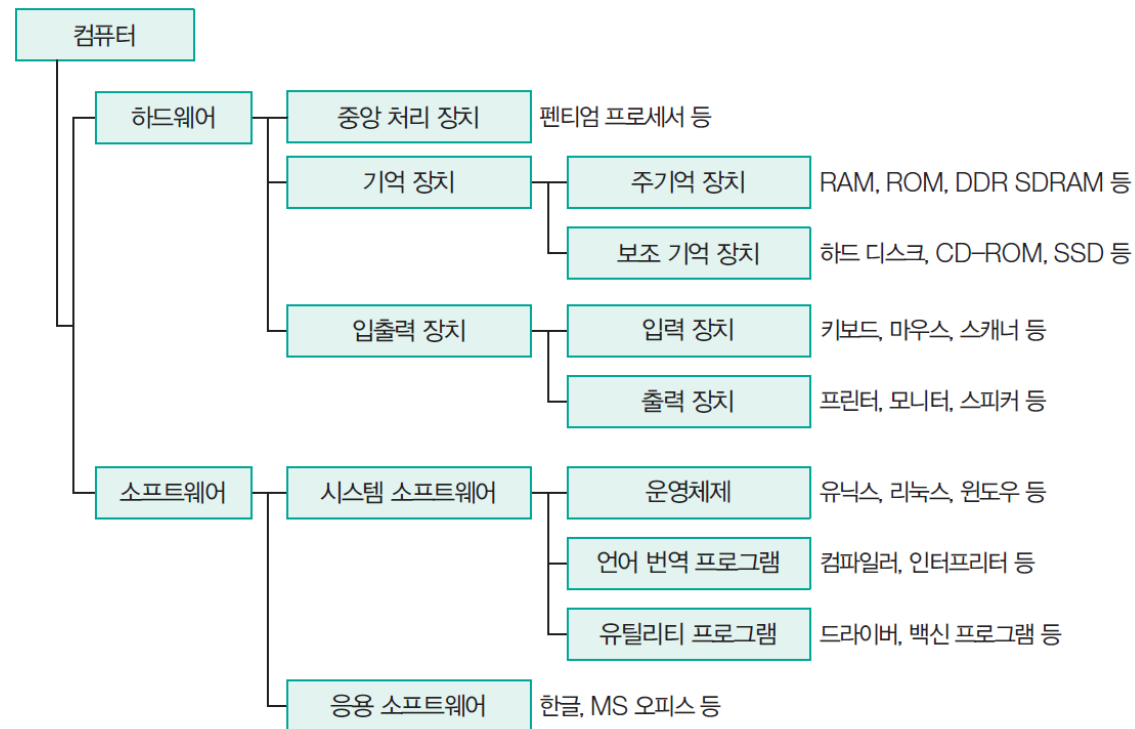


그림 1-1 컴퓨터의 기본 구성

# 컴퓨터 시스템의 구성

## 1 하드웨어

- 컴퓨터 하드웨어는 중앙 처리 장치, 주기억 장치, 보조 기억 장치, 입력 장치, 출력 장치, 시스템 버스로 구성된다.
- 컴퓨터의 기능을 수행하기 위해 각 구성 요소들은 시스템 버스를 통해 상호 연결되어 있다.

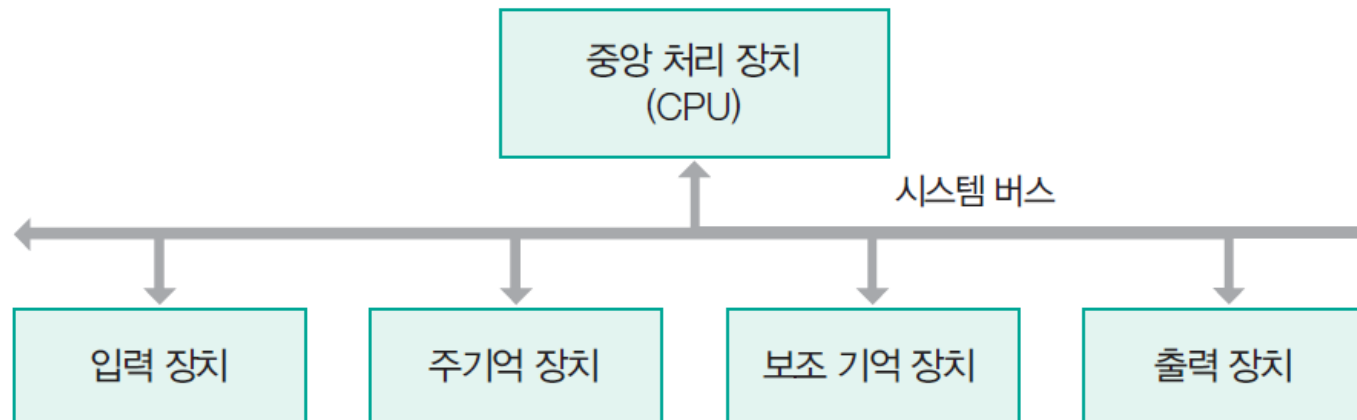
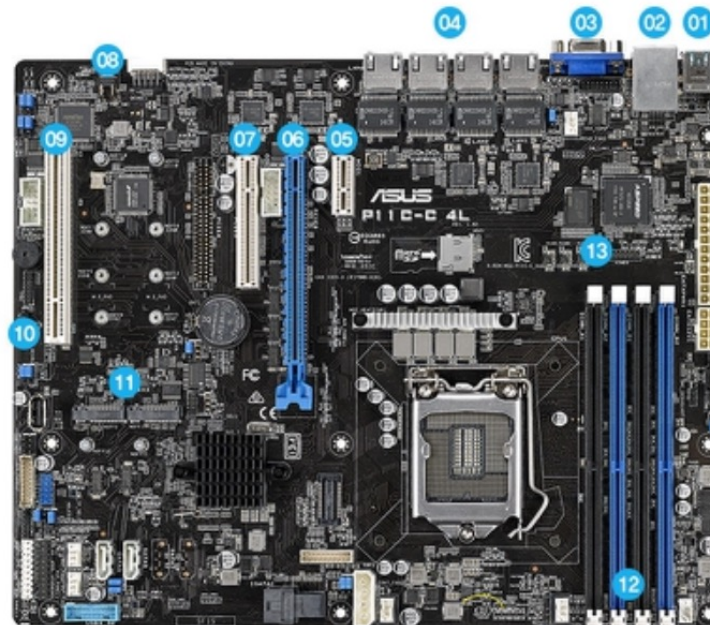


그림 1-2 컴퓨터의 하드웨어 구성



우리는 컴퓨터공학부 (일반,편입,전과,복수 등) 전공자이므로  
어디가 무엇인지 어떻게 연결되는지 알아야겠죠!



01. 2 x USB 3.1  
02. 2 x USB 3.0, 1 x Management LAN  
03. 1 x VGA port  
04. Quad Intel® server LAN  
05. 1 x MIO slot  
06. 1 x PCI-E x 16 (Gen3 x16 link)  
07. 1 x PCI-E x 8 (Gen3 x8 link)

08. Rear power button  
09. 1 x PCI 32bit/33 MHz  
10. 1 x TPM header  
11. 2 x M.2 (NGFF), 22110/2280/2260/2242  
12. 4 x DIMM slots  
13. Optional ASMB9-iKVM

# 컴퓨터 시스템의 구성

## □ 중앙 처리 장치

- **중앙 처리 장치**(Central Processing Unit, CPU) : 사실상 컴퓨터의 특성을 결정하며, 컴퓨터의 핵심 기능인 프로그램 실행과 데이터 처리를 담당한다.
- 중앙 처리 장치를 **프로세서**(processor) 또는 **마이크로프로세서**(microprocessor)라고도 부른다.
- **산술 논리 연산 장치**(Arithmetic and Logic Unit, ALU) : 산술 연산 논리 연산, 보수 연산, 시프트 연산을 수행한다.
- **제어 장치**(Control Unit, CU) : 프로그램의 명령어를 해독하여 명령어 실행에 필요한 제어 신호를 발생시키고 컴퓨터의 모든 장치를 제어한다.
- **레지스터**(register) : 중앙 처리 장치 내부에 있는 데이터를 일시적으로 보관하는 임시기억 장치로, 프로그램 실행 중에 사용되며 고속으로 액세스할 수 있다.

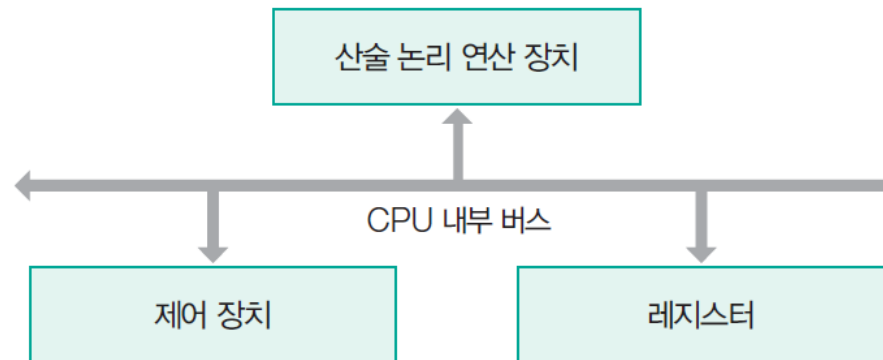


그림 1-3 중앙 처리 장치 내부 구조

# 컴퓨터 시스템의 구성

## □ 기억 장치

- **주 기억 장치**(main memory) : 반도체 칩으로 구성되어 고속으로 액세스가 가능하지만 고가다. 또한 프로그램 실행 중에 일시적으로만 사용되는 휘발성 메모리다.
- **보조 기억 장치**(auxiliary storage unit) : 하드 디스크나 SSD, CD-ROM 같은 비휘발성 메모리이며, 저장 밀도가 높고 저가이지만 속도가 느리다. 중앙 처리 장치에 당장 필요하지 않은 많은 양의 데이터나 프로그램을 저장하는 장치다.

## □ 입출력 장치

- **입력 장치**(input device) : 데이터를 전자적인 2진 형태로 변환하여 컴퓨터 내부로 전달한다. 키보드, 마우스가 있다.
- **출력 장치**(output device) : 중앙 처리 장치가 처리한 전자적인 형태의 데이터를 사람이 이해할 수 있는 데이터로 변환하여 출력한다. LCD 모니터, 프린터, 스피커가 있다.
- 입력 장치와 출력 장치를 합하여 **입출력 장치**(Input/Output device, I/O device)라고도 한다.

# 컴퓨터 시스템의 구성

## □ 시스템 버스

- **시스템 버스**(system bus) : 중앙 처리 장치와 기억 장치 및 입출력 장치 사이에 정보를 교환하는 통로이며 주소 버스, 데이터 버스, 제어 버스가 있다.

|                        |                                                                                                                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 주소 버스<br>(address bus) | <ul style="list-style-type: none"><li>• 기억 장치나 입출력 장치를 지정하는 주소 정보를 전송하는 신호선들의 집합이다.</li><li>• 단방향이다.</li></ul>                                                                      |
| 데이터 버스<br>(data bus)   | <ul style="list-style-type: none"><li>• 기억 장치나 입출력 장치 사이에 데이터를 전송하기 위한 신호선들의 집합이다.</li><li>• 데이터선의 수는 중앙 처리 장치가 한 번에 전송할 수 있는 데이터 비트의 수를 결정한다.</li><li>• 양방향 전송이 가능해야 한다.</li></ul> |
| 제어 버스<br>(control bus) | <ul style="list-style-type: none"><li>• 중앙 처리 장치가 시스템 내의 각종 요소의 동작을 제어하는 데 필요한 신호선들의 집합이다.</li><li>• 기억 장치 읽기와 쓰기 신호, 입출력 장치 읽기와 쓰기 신호 등이 있다.</li><li>• 단방향이다.</li></ul>            |



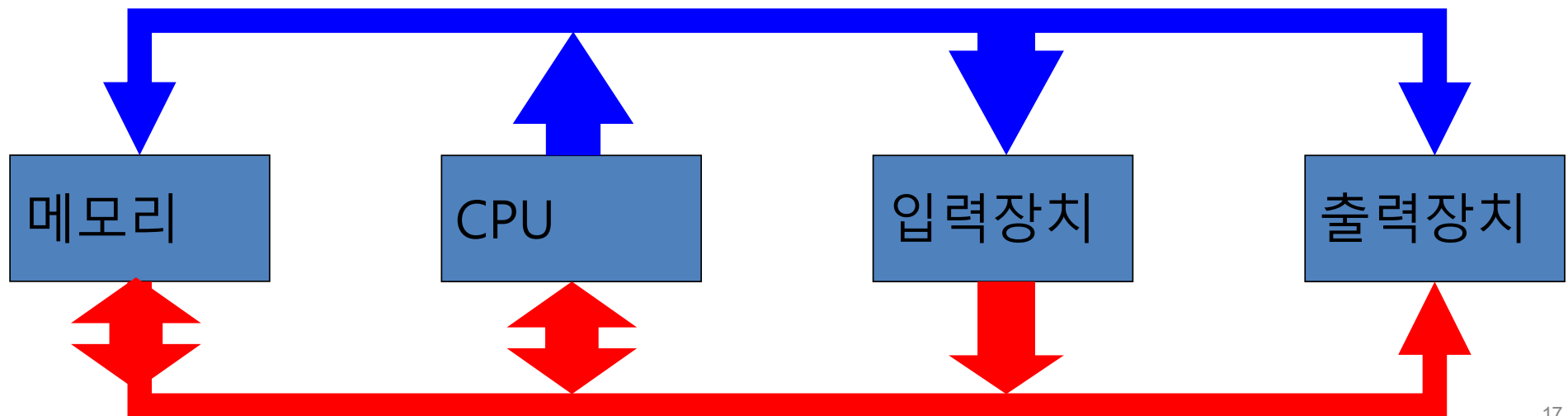
## BUS; 버스; 대량 이송 수단

### ■ Address Bus

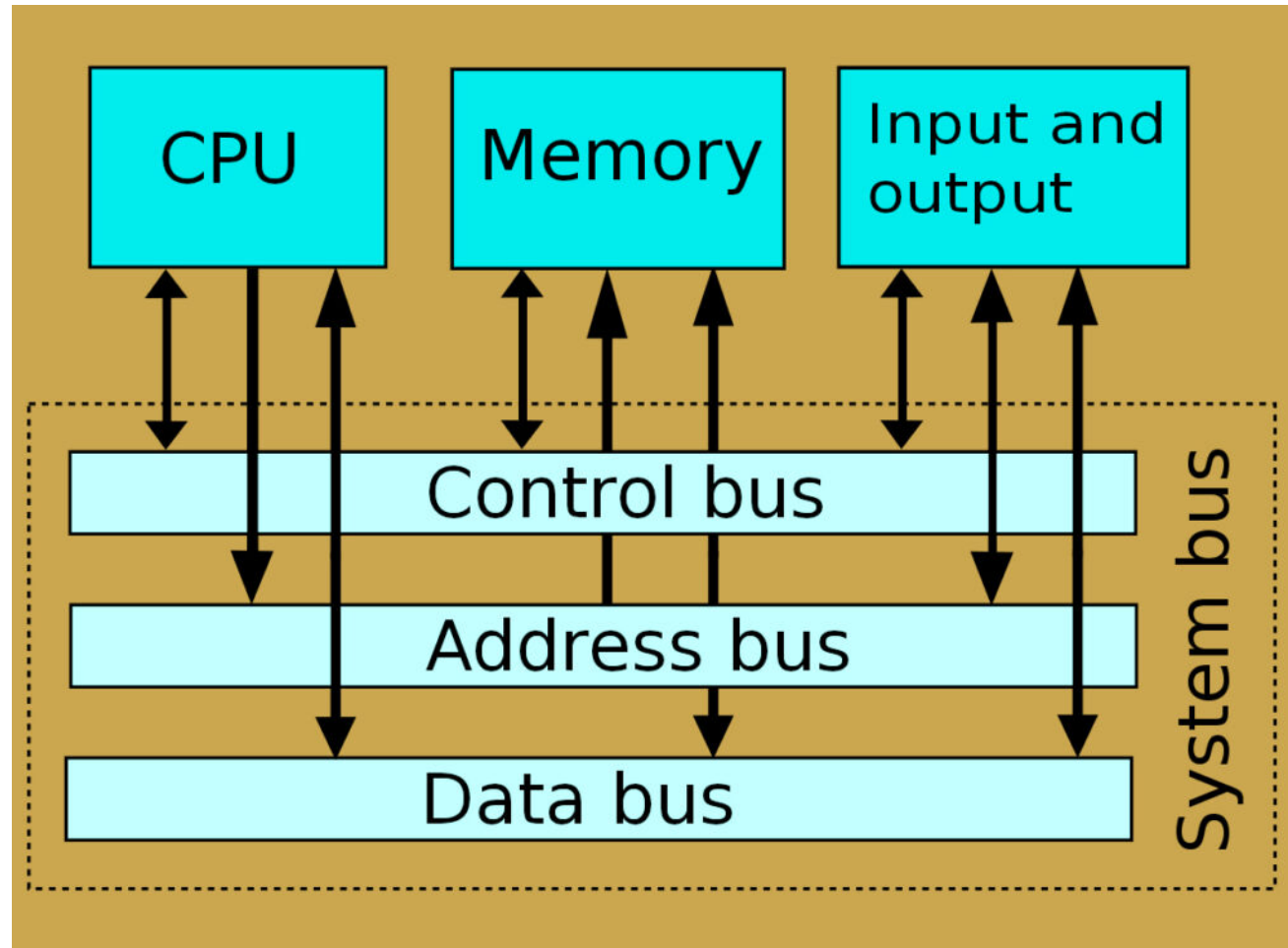
- CPU 및 각종 장치들의 Address Line을 연결

### ■ Data Bus

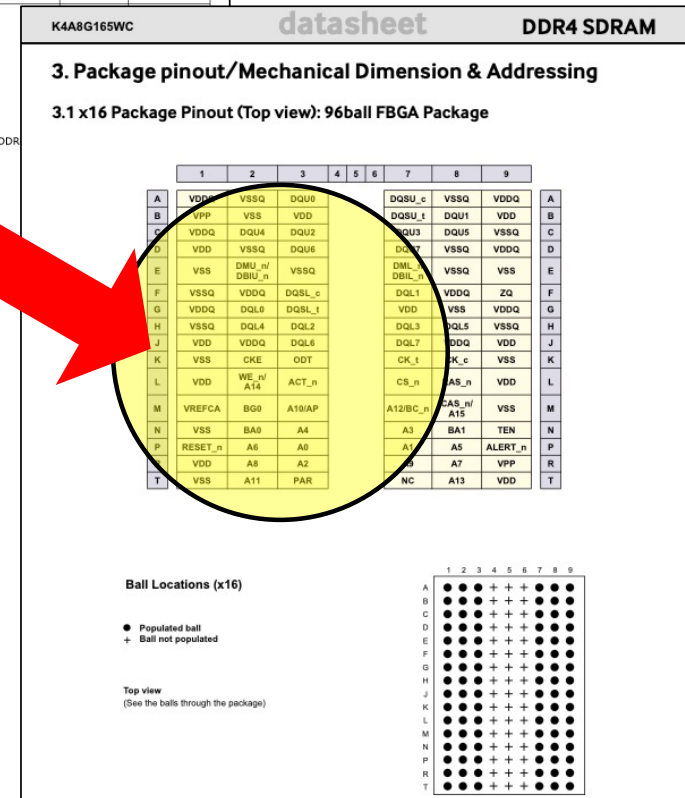
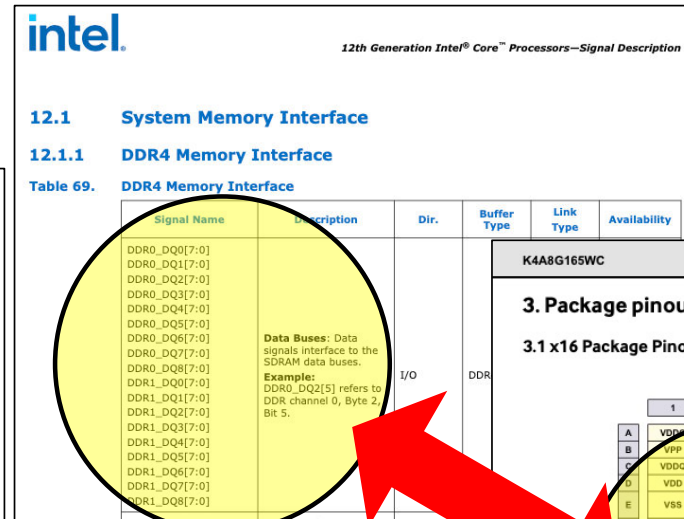
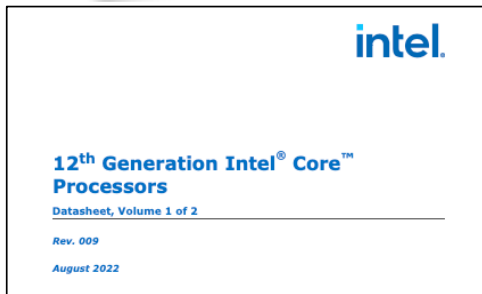
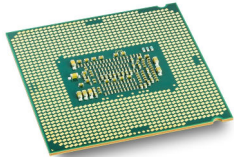
- CPU 및 각종 장치들의 Data Line을 연결



## BUS; 버스; 대량 이송 수단



# CPU와 Memory를 BUS로 연결



데이터시트(Datasheet)에 Pin Map을 보고  
CPU와 Memory의 해당 Pin들을 연결함  
이렇게 연결된 Pin들의 집합을 BUS라고 함  
\*메모리에 관련된 Pin들의 집합이면 메모리버스

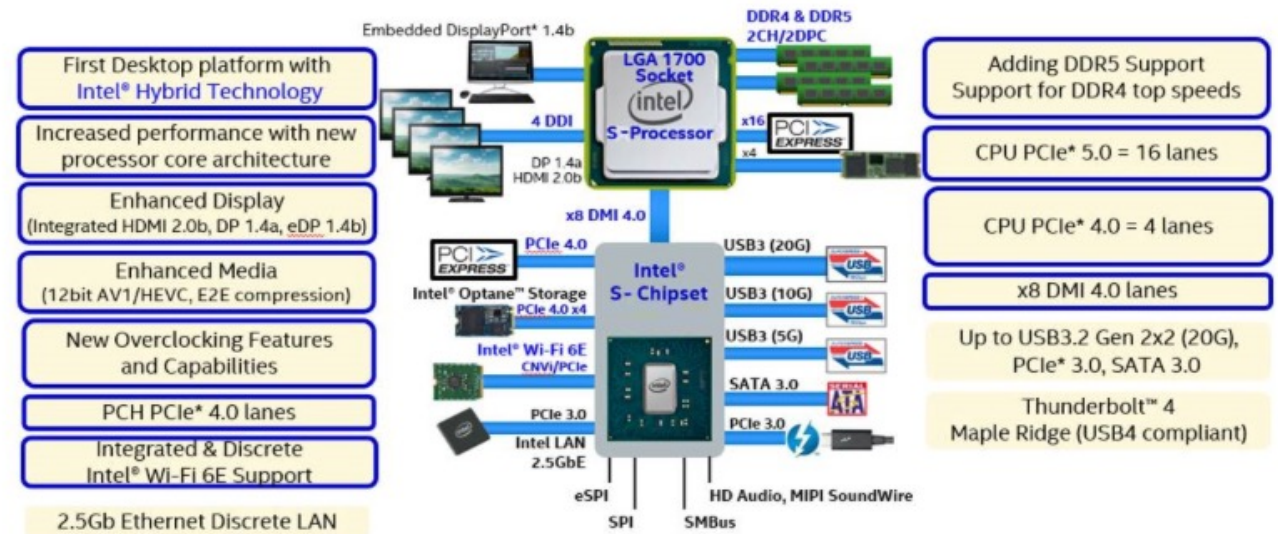
# BUS Control 필요함

메모리 여러개가 같은  
메모리버스를 공유함

따라서, 하나의 메모리가  
CPU랑 데이터를 주고 받을  
때, 버스에 연결된 다른  
메모리는 정지하고 있어야  
함. 그렇지 않으면 데이터  
충돌이 발생하게 됨

그래서, Clock에 동기하여  
Chip Enable, Read, Write  
신호를 통해서 버스를  
제어하게 됨

S Processor Line Platform Diagram



**Note:** Throughout this document, the 12<sup>th</sup> Generation Intel® Core™ Processors may be referred to as **processor** and the Intel® 600 Series Chipset Family Platform Controller Hub may be referred to as **PCH**.

# 컴퓨터 시스템의 구성

## □ 컴퓨터 시스템의 구성

- 주기억 장치는 중앙 처리 장치와 시스템 버스를 통해 연결된다.
- 보조 기억 장치나 입출력 장치는 속도가 느리기 때문에 인터페이스 회로나 제어기를 통해 중앙 처리 장치와 연결되어야 한다.

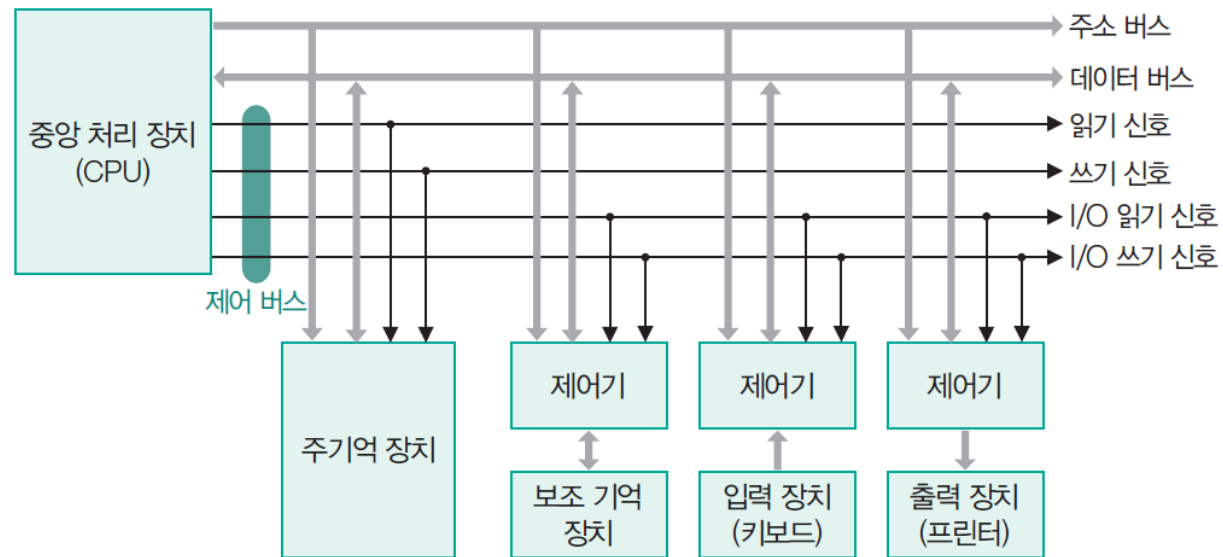


그림 1-4 중앙 처리 장치와 기억 장치 및 입출력 장치의 연결



# 컴퓨터 시스템의 구성

## 2 소프트웨어

- 소프트웨어 : 컴퓨터를 구성하고 있는 하드웨어를 잘 동작시킬 수 있도록 제어하고, 지시하는 모든 종류의 프로그램을 의미한다.
- 프로그램 : 컴퓨터를 사용해 어떤 일을 처리하기 위해 순차적으로 구성된 명령들의 집합이다.
- 소프트웨어는 시스템 소프트웨어와 응용 소프트웨어로 나눌 수 있다.

### □ 시스템 소프트웨어

- 시스템 소프트웨어(system software) : 하드웨어를 관리하고 응용 소프트웨어를 실행하는 데 필요한 프로그램이다.
- 운영체제(OS), 언어 번역 프로그램, 유틸리티 프로그램 등이 이에 속한다.
- 유틸리티 프로그램은 각종 주변 장치들을 구동하는 데 필요한 드라이버 프로그램, 백신 프로그램, TCP/IP 같이 컴퓨터를 네트워크로 연결하는 데 필요한 각종 프로그램 등을 말한다.

# 컴퓨터 시스템의 구성

## ■ 시스템 소프트웨어(계속)

|                            |                                                                                                                                                                                                                                                                                                                              |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 운영체제<br>(Operating System) | <ul style="list-style-type: none"><li>• 컴퓨터 하드웨어 자원인 중앙 처리 장치, 기억 장치, 입출력 장치, 네트워크 장치 등을 제어하고 관리하는 역할을 한다.</li><li>• 종류 : 유닉스 UNIX, 리눅스 LINUX, 윈도우 Windows, 맥 OS MAC OS, OS, 안드로이드 등</li></ul>                                                                                                                               |
| 언어 번역 프로그램                 | <ul style="list-style-type: none"><li>• 고급 언어 프로그램을 컴퓨터가 이해할 수 있는 기계어로 변환하는 프로그램</li><li>• <b>인터프리터</b>(interpreter)는 소스 프로그램을 한 줄씩 기계어로 번역하여 실행하기 때문에 실행 속도가 컴파일러보다 느리다. Basic이나 자바스크립트, HTML, SQL, Python 등</li><li>• <b>컴파일러</b>(compiler)는 전체 소스 프로그램을 한 번에 기계어로 직접 번역하여 실행하기 때문에 실행 속도가 빠르며, C, C++, C#, 자바 등</li></ul> |
| 장치 드라이버<br>(Device Driver) | <ul style="list-style-type: none"><li>• 컴퓨터에 온라인으로 연결된 주변 장치를 제어하는 운영체제 모듈을 말한다.</li></ul>                                                                                                                                                                                                                                   |
| 링커<br>(Linker)             | <ul style="list-style-type: none"><li>• 여러 개로 분할해 작성된 프로그램에 의해 생성된 목적 프로그램 또는 라이브러리 루틴을 결합하여 실행 가능한 하나의 프로그램으로 연결하는 프로그램</li><li>• 연결 편집기(Linkage Editor)라고도 한다.</li></ul>                                                                                                                                                   |
| 로더<br>(Loader)             | <ul style="list-style-type: none"><li>• 하드 디스크 같은 저장 장치에 보관된 프로그램을 읽어 주기억 장치에 적재한 후 실행 가능한 상태로 만드는 프로그램이다.</li><li>• 로더는 할당, 연결, 재배치, 적재 기능을 수행한다.</li></ul>                                                                                                                                                                 |

# 컴퓨터 시스템의 구성

## ■ 응용 소프트웨어

- 컴퓨터 시스템을 일반 사용자들이 특정한 용도에 활용하기 위해 만든 프로그램으로, 애플리케이션, 앱, 어플이라고도 한다.

표 1-1 응용 소프트웨어의 용도와 예

| 용도         | 예                                 |
|------------|-----------------------------------|
| 사무용        | 한글, MS-office 제품군                 |
| 그래픽용       | 포토샵, 페인트샵, 일러스트레이터                |
| 멀티미디어용     | WinAMP, GOM Player, PowerDVD      |
| 게임용        | 스타크래프트, 워크래프트, 퀘이크                |
| 통신 및 네트워크용 | 인터넷 익스플로러, 크롬, 모질라, MSN 메신저, 카카오톡 |

# 컴퓨터 시스템의 구성

## 3 프로그램 처리 과정

- 프로그램은 고급 언어 → 어셈블리어 → 기계어 순으로 변환



그림 1-5 고급 언어 프로그램의 변환 과정과 예

# 컴퓨터 시스템의 구성

## ❖ 기계어 구조 예

### ① LOAD A, X

- 메모리 10번지의 내용을 읽어 레지스터 A에 로드(LOAD)하라는 의미

### ② ADD A, Y

- 레지스터 A의 내용과 메모리 11번지의 내용을 더해 그 결과를 레지스터 A에 로드하라는 의미

### ③ STOR Z, A

- 레지스터 A의 내용을 메모리 12번지에 저장하라는 의미

|           | 연산 코드 | 오퍼랜드  |
|-----------|-------|-------|
| LOAD A, X | 001   | 01010 |
| ADD A, Y  | 011   | 01011 |
| STOR Z, A | 010   | 01100 |
|           | 3비트   | 5비트   |

그림 1-6 기계어 구조 예

## ❖ 연산 코드(opcode)

- CPU가 수행할 연산을 지정하는 비트들
- 비트 수 = 3이면, 지정할 수 있는 연산의 최대 수 :  $2^3 = 8$ 개

## ❖ 오퍼랜드(operand)

- 연산에 사용될 데이터가 저장되어 있는 기억 장치 주소
- 비트 수 = 5 이면, 주소지정할 수 있는 기억 장치의 최대 수 :  $2^5 = 32$  개

## 컴퓨터 시스템의 구성

### ❖ 프로그램과 데이터가 주기억 장치에 저장되어 있는 형태

- 명령어와 데이터는 지정된 기억 장소에 저장
- 워드(word) 단위로 저장
- 워드는 CPU에 의해 한 번에 처리될 수 있는 비트들의 그룹 (8, 16, 32, 66비트)

| 주소 |          |
|----|----------|
| 0  | 00101010 |
| 1  | 01101011 |
| 2  | 01001100 |
| 3  | ...      |
| 4  | ...      |
| 5  | ...      |
| 6  | ...      |
| 7  | ...      |
| 8  | ...      |
| 9  | ...      |
| 10 | 10010110 |
| 11 | 00011110 |
| 12 | ...      |
| 13 | ...      |

명령어

데이터

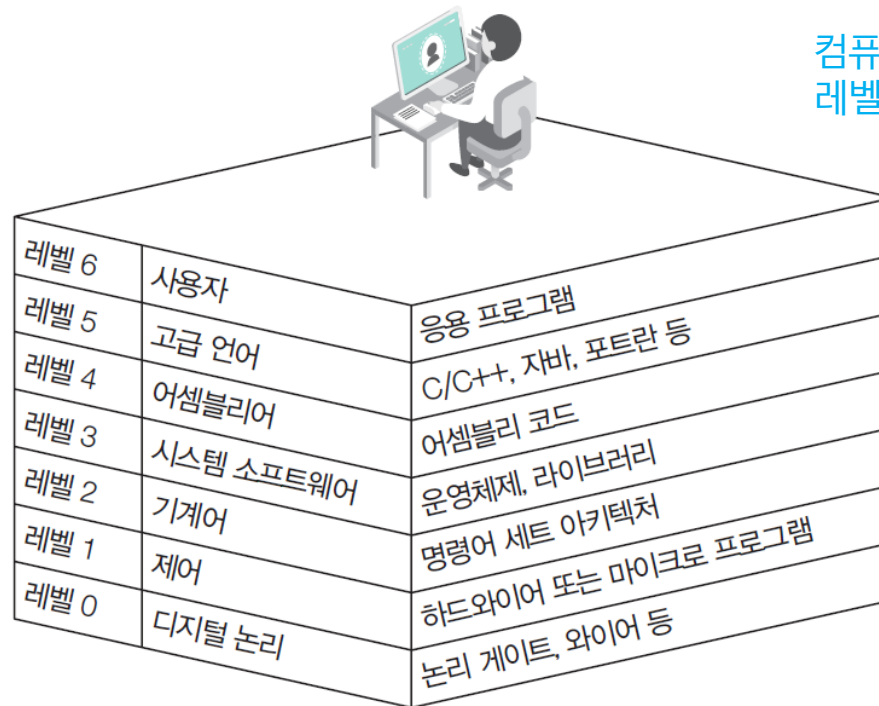
그림 1-7 프로그램과 데이터가 주기억 장치에 저장되어 있는 예



# 컴퓨터 시스템의 구성

## 4 컴퓨터 시스템의 계층 레벨

- **레벨 6** : 사용자 레벨이며 응용 프로그램들로 구성된다. 이 레벨에서 사용자들은 워드 프로세서, 그래픽 패키지, 게임 등과 같은 프로그램을 동작시킨다.
- **레벨 5** : C/C++, JAVA, FORTRAN 같은 고급 언어로 구성되어 있다. 고급 언어 → 어셈블리어 → 기계어 순으로 변환된다.



컴퓨터 구조에서는  
레벨 0부터 레벨 2까지 다룬다.

그림 1-8 컴퓨터 시스템의 추상화 레벨

# 컴퓨터 시스템의 구성

- **레벨 4** : 어셈블리어를 포함한다. 어셈블리어 명령어 1개가 정확히 기계어 명령어 1개로 변환된다.
- **레벨 3** : 시스템 소프트웨어 레벨로 운영체제와 라이브러리들로 구성된다. 이 레벨은 다중 프로그래밍, 메모리 할당, 프로세스 관리와 기타 중요 기능을 담당한다.
- **레벨 2** : 명령어 세트 아키텍처(Instruction Set Architecture, ISA) 레벨인 컴퓨터 시스템의 특정 구조에 의해 인식되는 기계어로 구성되어 있다.
- **레벨 1**은 제어 레벨로 제어 장치가 명령어들을 해독(decode)하고 실행하며, 데이터들을 이동할 시간과 장소들을 결정한다. 제어 장치는 레벨 2에서 기계어 명령어를 한번에 하나씩 읽어서 필요한 동작을 수행하기 위한 신호들을 발생시킨다. 제어 장치는 하드와이어 hardwired 방식과 마이크로 프로그램 microprogrammed 방식 중 하나로 구현할 수 있다(5장에서 설명).

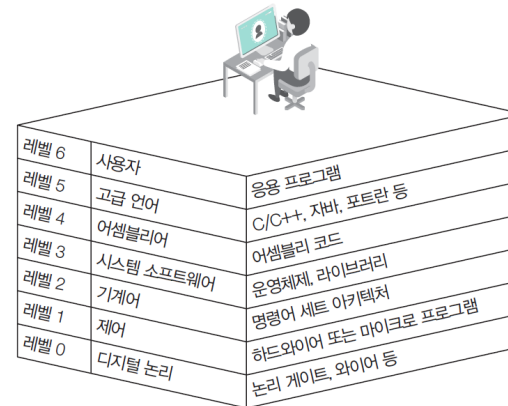
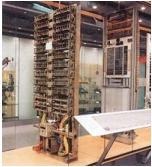


그림 1-8 컴퓨터 시스템의 추상화 레벨

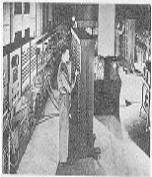
# 컴퓨터의 역사

## □ 전기 기계식 계산기



- **MARK-I** : 1944년 하버드 대학의 에이킨이 개발. MARK-I은 배비지의 해석 기관을 실현시킨 것으로서 미해군의 탄도 계산에 이용되었으며, 수많은 수학이나 과학 문제를 해결하는 데 공헌했다.

## □ 전자식 계산기



- **에니악**(ENIAC, Electronic Numerical Integrator And Computer) : 진공관을 사용한 최초의 전자식 컴퓨터(1946년, 에커트와 모클리)
- **에드삭**(EDSAC, Electronic Delay Storage Automatic Calculator) : 10진수 체계와 프로그램 내장 방식의 계산기(1949년, 윌키스)
- **에드박**(EDVAC, Electronic Discrete Variable Automatic Computer) : 2진수 체계와 프로그램 내장 방식을 적용(1951년, 폰 노이만)
- **유니박**(UNIVAC, UNIVersal Automatic Computer) : 최초의 상용 컴퓨터, 인구조사통계국에 설치(1951년, 모클리와 에커트)

컴퓨터의 발전 과정에 있어서 진공관 및 프로그램 내장 방식의 사용은 근대에서 현대로 넘어 오게 되는 분기점이 되었다.

# 컴퓨터의 역사

## 2 컴퓨터의 세대별 분류

- 컴퓨터의 발전 과정을 세대별로 명확하게 설명하기는 어렵지만 새로운 하드웨어 부품의 출현을 기준으로 분류되고 있다.

표 1-2 컴퓨터 세대 구분

| 구분 \ 세대  | 1세대                | 2세대                                | 3세대                                    | 4세대                            | 5세대                   |
|----------|--------------------|------------------------------------|----------------------------------------|--------------------------------|-----------------------|
| 주요 소자    | 진공관                | 트랜지스터                              | SSI, MSI                               | LSI, VLSI                      | VLSI, ULSI            |
| 주 기억 장치  | 자기 드럼,<br>수은 지연 회로 | 자기 코어                              | IC(RAM, ROM)                           | LSI, VLSI                      | VLSI                  |
| 보조 기억 장치 | 천공 카드,<br>종이 테이프   | 자기 드럼,<br>자기 디스크                   | 자기 디스크,<br>자기 테이프                      | 자기 디스크,<br>자기 테이프              | 자기 디스크,<br>광 디스크      |
| 처리 속도    | ms( $10^{-3}$ )    | $\mu$ s( $10^{-6}$ )               | ns( $10^{-9}$ )                        | ps( $10^{-12}$ )               | fs( $10^{-15}$ )      |
| 사용 언어    | 기계어,<br>어셈블리어      | 고급 언어(COBOL,<br>FORTRAN,<br>ALGOL) | 고급 언어(LISP,<br>PASCAL, BASIC,<br>PL/I) | 고급 언어<br>(ADA 등),<br>문제 지향적 언어 | 객체 지향 언어<br>(C++, 자바) |



릴레이



진공관



트랜지스터



IC

# 컴퓨터의 역사

## ❖ 집적 회로의 분류

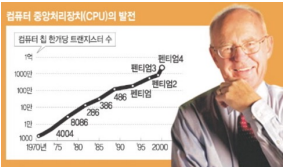
- 집적 회로(Integrated Circuit, IC)가 개발된 3세대 이후에는 컴퓨터 세대에 대한 정의가 분명하지 않다.

|                                |                                                                                                                                                                               |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SSI<br>(Small Scale IC)        | <ul style="list-style-type: none"><li>• 트랜지스터 수십 개가 집적된 소규모 IC로, 기본 게이트 기능과 플립플롭이 이에 해당한다.</li></ul>                                                                          |
| MSI<br>(Medium Scale IC)       | <ul style="list-style-type: none"><li>• 트랜지스터 수백 개가 집적된 중규모 IC로, 디코더, 인코더, 멀티플렉서, 디멀티플렉서, 카운터, 레지스터, 소형 기억 장치 등의 기능을 포함한다.</li></ul>                                          |
| LSI<br>(Large Scale IC)        | <ul style="list-style-type: none"><li>• 트랜지스터 수천 개가 집적된 대규모 IC로, 8비트 마이크로프로세서나 소규모 반도체 기억 장치 칩이 이에 해당한다.</li></ul>                                                            |
| VLSI<br>(Very Large Scale IC)  | <ul style="list-style-type: none"><li>• 트랜지스터 수만에서 수십만 개 이상 집적된 초대규모 IC로, 대용량 반도체 메모리, 1만 게이트 이상의 논리 회로, 단일 칩 마이크로프로세서 등이 있다.</li></ul>                                       |
| ULSI<br>(Ultra Large Scale IC) | <ul style="list-style-type: none"><li>• 트랜지스터가 수백만 개 이상 집적된 극대규모 IC로, 인텔의 486이나 팬티엄, 수백 메가바이트 이상의 반도체 기억 장치 칩 등이 이에 해당한다.</li><li>• VLSI와 ULSI 사이의 정확한 구분은 사실 모호하다.</li></ul> |

# 컴퓨터의 역사

## 3 무어의 법칙과 황의 법칙

### □ 무어의 법칙



- 반도체 집적 회로의 **트랜지스터 수가 2년(24개월)마다 2배로 증가한다는 법칙**이다.
- 1970년대 중반부터 이 기간은 12개월에서 18개월로 늘어났다. 이럴 경우에 10년 동안 증가율은 1,000배가 아니라 100배 정도가 되었다. 1990년대부터는 18개월이 다시 24개월로 늘어났는데, 이 경우에 10년간 증가율은 32배 정도로 줄어들었다.
- 무어의 법칙은 한계에 다다르고 있다. 이것은 기술적인 한계뿐만 아니라 경제적인 한계도 있었다.

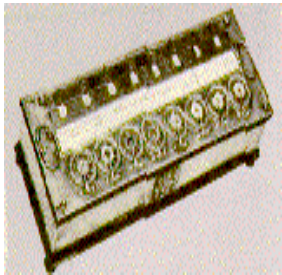
### □ 황의 법칙



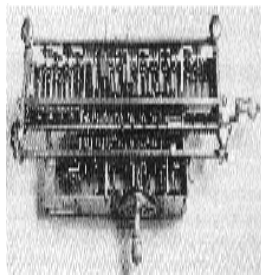
- 집적 회로를 뛰어넘는 메모리의 발전으로 인해서 앞으로는 **1년에 2배씩 용량이 뛰어오를 것**이라고 주장한 것
- 이는 계속해서 NAND flash 계열의 메모리가 지속적으로 발전하면서 실제로 증명되었다. 하지만 2010년, 불과 8년 만에 황의 법칙은 깨졌다.



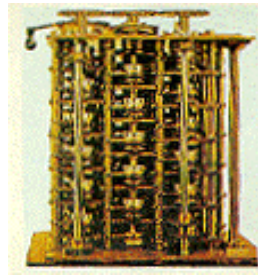
# 계산기부터 컴퓨터까지



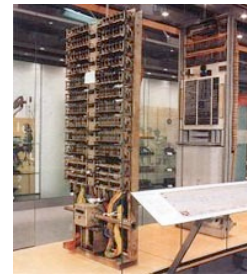
파스칼 계산기  
1642년



라이프니츠 계산기  
1671년



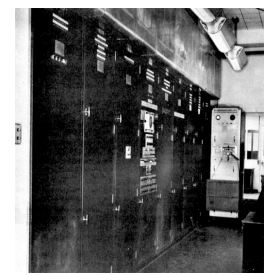
베비지 차등분석기  
1823년



MARK-I  
1944년



ENIAC  
1946년



EDVAC(폰 노이만)  
1951년



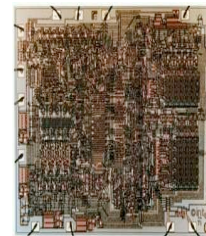
UNIVAC  
1951년



DEC PDP-1  
1957년



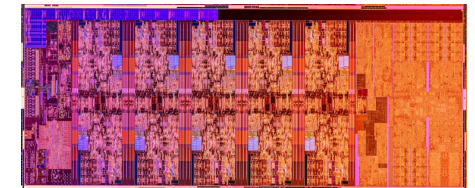
IBM 7904  
1959년



Intel 4004  
1971년



Apple II  
1980년



Intel Core-i9  
2020년



# 폰 노이만, 비 폰 노이만, 하버드 구조

## 1 폰 노이만 구조와 비 폰 노이만 구조

- 사실 폰 노이만 구조 개념을 처음으로 생각한 사람은 모클리와 에커트였다.
- 폰 노이만 구조의 프로그램 처리 과정
  - ① 프로그램 카운터를 이용해 메모리에서 실행할 명령어를 인출한다.
  - ② 제어 장치는 이 명령어를 해독한다.
  - ③ 명령을 실행하는 데 필요한 데이터를 메모리에서 인출하여 레지스터에 저장한다.
  - ④ 산술 논리 연산 장치는 명령을 실행하고, 레지스터나 메모리에 결과를 저장한다.

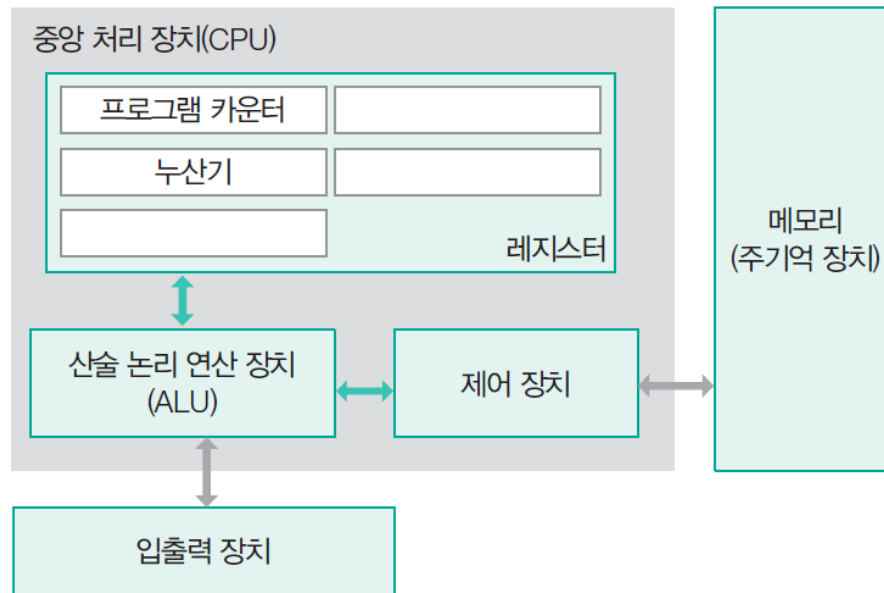


그림 1-12 폰 노이만 구조

## 폰 노이만, 비 폰 노이만, 하버드 구조

### ❖ 폰 노이만 병목 현상(Von-Neumann Bottleneck)

- 폰 노이만 구조의 컴퓨터는 중앙 처리 장치에서 명령어나 데이터를 메모리에서 가져와 처리한 후, 결과 데이터를 메모리에 다시 보내 저장한다. 또한 저장된 데이터가 필요할 땐 다시 메모리에서 중앙 처리 장치로 불러오는 방식으로 **순차적으로 프로그램을 처리하므로 메모리나 시스템 버스에 병목 현상이 생겨 속도가 느려질 수밖에 없다.**
- 이것을 데이터 경로의 병목 현상 또는 기억 장소의 지연 현상이라고도 한다.

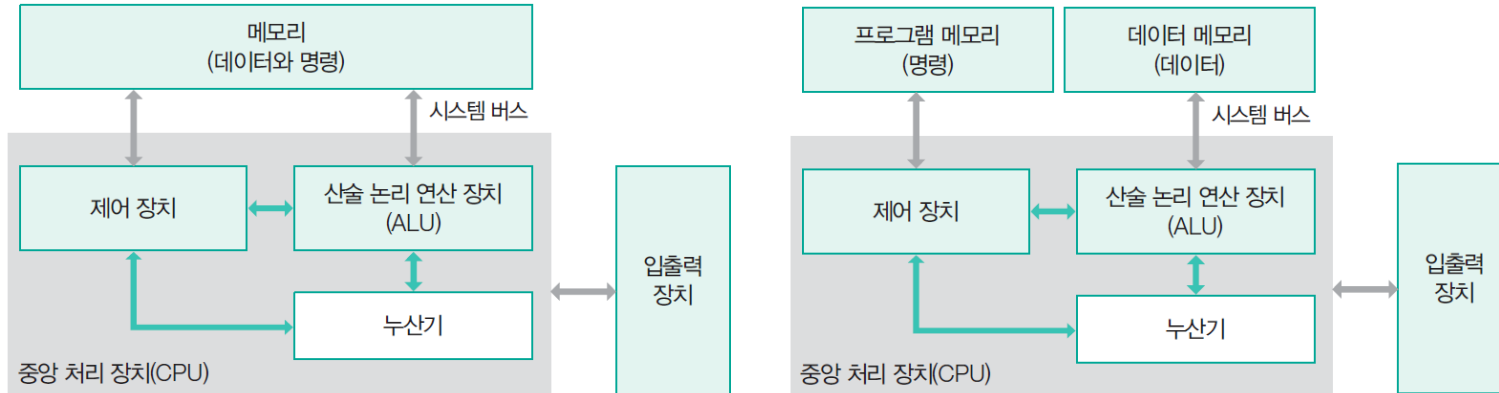
### ❖ 비 폰 노이만 구조

- 폰 노이만 구조가 아닌 컴퓨터를 통틀어 이르는 말로 데이터 처리의 고속화와 고도화를 위하여 프로그램 일부를 하드웨어화하거나, 병렬 처리 기능, 추론 기구를 채택한 컴퓨터를 가리킨다.
- 연구되고 있는 분야로는 신경망, 유전 알고리즘, 양자 컴퓨터, 병렬 컴퓨터를 비롯한 여러 분야들이 비 폰 노이만 범주에 속한다. 이 중에서 병렬 컴퓨팅이 현재 가장 많이 사용된다.

# 폰 노이만, 비 폰 노이만, 하버드 구조

## 2 폰 노이만 구조와 하버드 구조

- 폰 노이만 구조는 구조적으로 시스템 버스에 병목 현상이 발생한다.
- 하버드 구조는 폰 노이만 구조의 단점을 보완한 개념이다.
- 명령어 메모리 영역과 데이터 메모리 영역을 물리적으로 분리시키고, 각각을 다른 시스템 버스로 중앙 처리 장치에 연결함으로써 명령과 데이터를 메모리로부터 읽는 것을 동시에 처리할 수 있다.
- 그러나 하버드 구조는 비싸고, 공간도 많이 차지하며, 설계가 복잡하다.
- 최근 CPU 설계에서는 폰 노이만 구조와 하버드 구조 양자의 구조를 도입하고 있다.



(a) 폰 노이만 구조

(b) 하버드 구조

그림 1-13 폰 노이만 구조와 하버드 구조

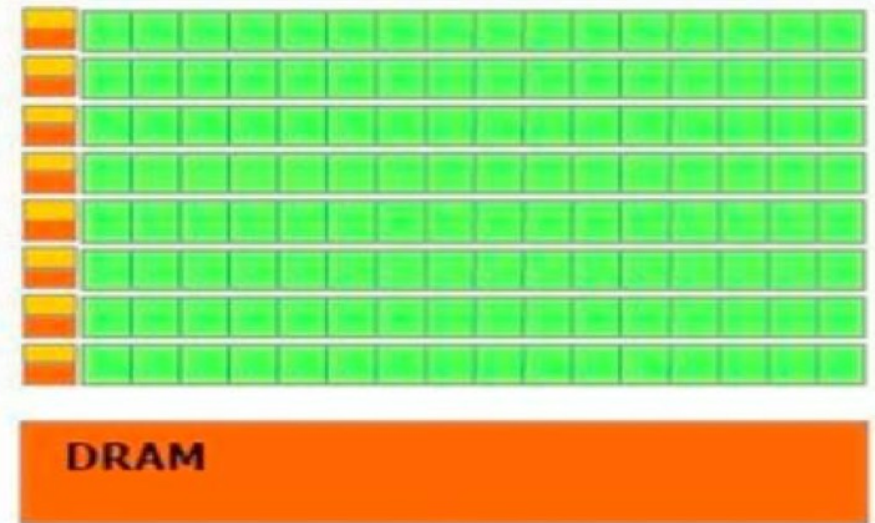
# CPU vs GPU

컴퓨터의 전체를 제어하는 CPU와 특정 계산만 하는 GPU의 구조 차이



**CPU**

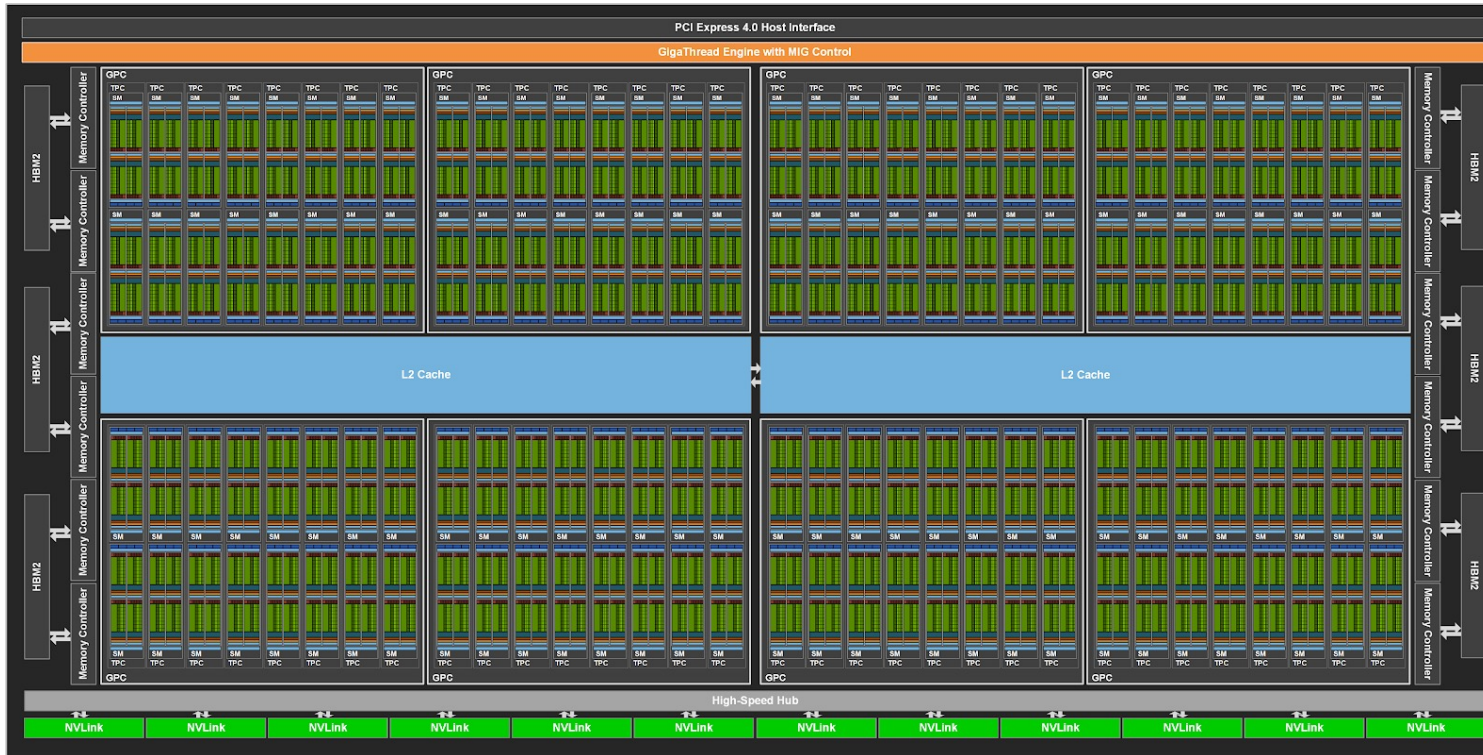
- 순차 처리에 특화
- Cache 메모리(RAM과의 속도차이)



**GPU**

- 병렬 처리에 특화
- 연산할수 있는 ALU이 많음

# NVIDIA Ampere Architecture



그 어떤 학교나 학원에서 배울 수 없는  
현장 경험을 공유하며

컴퓨터구조를 꿰뚫고 있는  
실력 있는 개발자가 될 수 있도록 교육하겠습니다.

감사합니다.



e-mail: [hertzkim@dau.ac.kr](mailto:hertzkim@dau.ac.kr)

phone: 051-200-7928

office: s06-0626